

SERVICIO NACIONAL DE APRENDIZAJE — SENA

Programa: Análisis y Desarrollo de Software

Proyecto Formativo: Construcción de Software Integrador de Tecnologías Orientadas a Servicios

EVIDENCIA DE DESEMPEÑO

GA7-220501096-AA5-EV01

Diseño y Desarrollo de Servicios Web

Proyecto: Sistema Integral de Gestión de Leads

Campo	Dato
Aprendiz	JULIAN ENRIQUE OCAMPO LOPEZ
Ficha ADSO	3118307
Actividad	GA7-220501096-AA5 — diseño y desarrollo de servicios web-proyecto
Repositorio GitHub	https://github.com/JULIAN04301969/leads-auth-api
Área	TÉCNICA
Instructor	Ing. HENRY ALFONSO GARZÓN SÁNCHEZ
Fecha de entrega	abril de 2026

1. Introducción

El presente documento registra el diseño, desarrollo e implementación del servicio web de autenticación correspondiente a la evidencia de desempeño GA7-220501096-AA5-EV01, perteneciente a la Actividad de Proyecto 7 de la Fase 3 de Ejecución del programa de formación Análisis y Desarrollo de Software del SENA.

El proyecto marco en el que se inscribe esta evidencia es el Sistema de Gestión Integral de Leads, un software orientado a la administración del ciclo de vida de oportunidades comerciales. Dentro de este sistema, el control de acceso constituye un componente crítico, ya que garantiza que únicamente usuarios autorizados puedan interactuar con la información del negocio.

El servicio desarrollado expone dos operaciones fundamentales mediante una API REST:

- Registro de usuarios: permite crear nuevas cuentas de acceso al sistema con nombre, correo electrónico, contraseña y rol asignado.
- Inicio de sesión: valida las credenciales del usuario y, si son correctas, devuelve un mensaje de autenticación satisfactoria junto con un token JWT. En caso contrario, retorna un mensaje de error en la autenticación.

El desarrollo se realizó con las herramientas tecnológicas seleccionadas para el proyecto: Visual Studio Code como entorno de desarrollo integrado, Node.js con Express como plataforma y framework de la API, MySQL con MySQL Workbench para la gestión de la base de datos, y GitHub como repositorio de control de versiones.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un servicio web de autenticación para el Sistema de Gestión Integral de Leads, que permita el registro seguro de usuarios y la validación de credenciales mediante inicio de sesión, aplicando buenas prácticas de desarrollo de API REST y seguridad en el manejo de contraseñas.

2.2 Objetivos específicos

1. Construir un endpoint de registro que reciba, valide y persista los datos de nuevos usuarios en la base de datos MySQL, almacenando la contraseña mediante hash bcrypt.
2. Implementar un endpoint de inicio de sesión que compare las credenciales ingresadas con los registros almacenados y genere un token JWT firmado ante una autenticación satisfactoria.
3. Desarrollar un middleware de verificación de tokens JWT para proteger rutas privadas del sistema.
4. Aplicar validaciones de entrada que garanticen la integridad de los datos recibidos por el servicio.
5. Publicar el código fuente en un repositorio GitHub como evidencia del uso de herramientas de versionamiento de software.

3. Marco tecnológico

3.1 API REST

Una API (Application Programming Interface) REST es un conjunto de convenciones que permiten la comunicación entre sistemas a través del protocolo HTTP. Cada recurso del sistema es accesible mediante una URL única y las operaciones se expresan con los verbos HTTP estándar: GET, POST, PUT y DELETE. Las respuestas se entregan en formato JSON, estándar de intercambio de datos compatible con prácticamente todos los lenguajes de programación modernos.

3.2 Node.js y Express

Node.js es un entorno de ejecución JavaScript del lado del servidor, basado en el motor V8 de Chrome. Su modelo de entrada/salida no bloqueante lo hace especialmente eficiente para aplicaciones de red con múltiples conexiones concurrentes. Express es el framework web más utilizado sobre Node.js; simplifica la definición de rutas, el uso de middlewares y la gestión de peticiones y respuestas HTTP.

3.3 JSON Web Tokens (JWT)

Un JSON Web Token es un estándar abierto (RFC 7519) que define un método compacto y autocontenido para transmitir información entre partes como un objeto JSON firmado digitalmente. En el contexto de autenticación, el servidor genera un token después de validar las credenciales y lo envía al cliente. Las peticiones posteriores incluyen este token, permitiendo al servidor verificar la identidad sin consultar la base de datos en cada operación.

3.4 Hashing con bcrypt

bcrypt es un algoritmo de hashing adaptativo diseñado para el almacenamiento seguro de contraseñas. A diferencia de los algoritmos de hash general como MD5 o SHA-1, bcrypt incluye un factor de costo configurable que ralentiza intencionalmente el cómputo, haciendo los ataques de fuerza bruta computacionalmente inviables. Nunca se almacena la contraseña en texto plano; solo su hash irreversible.

3.5 MySQL

MySQL es un sistema de gestión de bases de datos relacionales de código abierto, ampliamente utilizado en aplicaciones web. En este proyecto se emplea para persistir los datos de los usuarios del sistema, incluyendo sus credenciales de acceso en forma de hash. La comunicación con MySQL desde Node.js se realiza a través de la librería mysql2, que ofrece soporte nativo para operaciones asíncronas con async/await.

4. Arquitectura del servicio

El servicio sigue el patrón de arquitectura en capas, que separa las responsabilidades en módulos independientes y cohesivos, facilitando el mantenimiento y la escalabilidad del sistema:

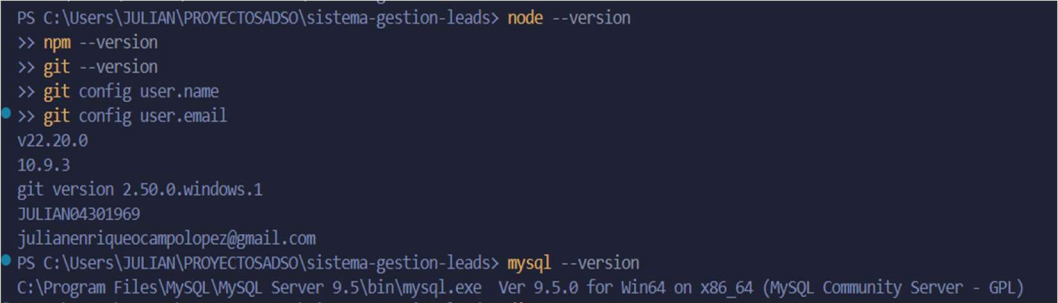
Capa	Archivo	Responsabilidad
Entrada	src/index.js	Configuración del servidor, middlewares globales y arranque del proceso.
Enrutamiento	src/routes/authRoutes.js	Mapeo de URLs a controladores. Define endpoints públicos y privados.
Control	src/controllers/authController.js	Lógica de negocio: validaciones, operaciones en BD y generación de tokens.
Seguridad	src/middleware/authMiddleware.js	Verificación y decodificación del token JWT en rutas protegidas.
Datos	src/config/database.js	Pool de conexiones MySQL reutilizables para todas las operaciones.
Cliente	public/index.html	Interfaz web para probar el API desde el navegador sin herramientas externas.

Los endpoints disponibles son:

Método	Ruta	Acceso	Descripción
POST	/api/auth/registro	Público	Crea un nuevo usuario. Hashea la contraseña con bcrypt.
POST	/api/auth/login	Público	Valida credenciales y emite token JWT si son correctas.
GET	/api/auth/perfil	Privado	Datos del usuario autenticado. Requiere token JWT.
GET	/api/estado	Público	Health check del servidor y la base de datos.

5. Desarrollo de la evidencia

A continuación, se describe el proceso completo de implementación del servicio web, paso a paso, con la captura de pantalla correspondiente a cada etapa.

Paso 1 — Verificación del entorno de desarrollo	
Descripción	Se verifica que Node.js v22.20.0, npm 10.9.3, Git 2.50.0 y MySQL 9.5.0 están instalados y operativos. Esta verificación garantiza un entorno estable antes de iniciar el desarrollo.
Acción	<code>node --version</code> <code>npm --version</code> <code>git --version</code> <code>mysql --version</code>
Captura 1	CAPTURA 1 — VERIFICACIÓN DEL ENTORNO DE DESARROLLO 

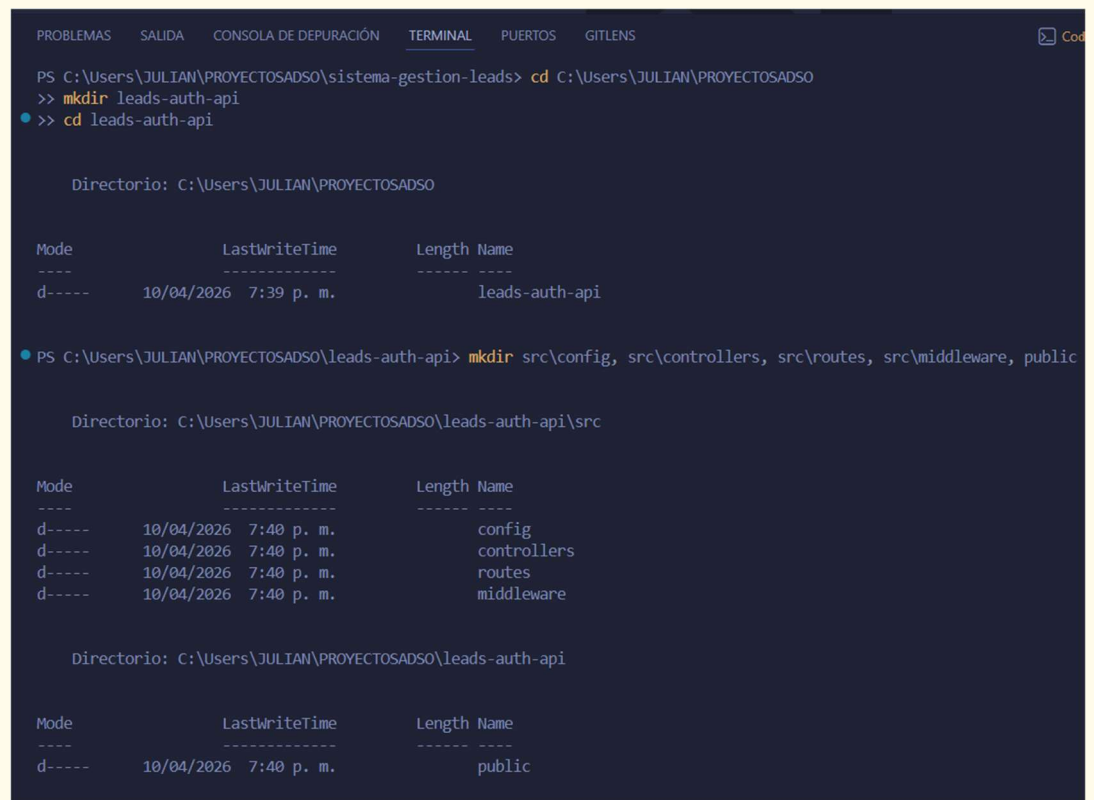
Paso 2 — Creación de la estructura del proyecto

Descripción Se crea la carpeta leads-auth-api en C:\Users\JULIAN\PROYECTOSADSO\ y la estructura de directorios que organiza el código fuente por capas: config, controllers, routes, middleware y public.

Acción `mkdir leads-auth-api && cd leads-auth-api && mkdir src\config src\controllers src\routes src\middleware public`

Captura 2

CAPTURA 2 — CREACIÓN DE LA ESTRUCTURA DEL PROYECTO



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  GITLENS  Cod

PS C:\Users\JULIAN\PROYECTOSADSO\sistema-gestion-leads> cd C:\Users\JULIAN\PROYECTOSADSO
>> mkdir leads-auth-api
● >> cd leads-auth-api

Directorio: C:\Users\JULIAN\PROYECTOSADSO

Mode                LastWriteTime         Length Name
----                -
d-----         10/04/2026   7:39 p. m.         leads-auth-api

● PS C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api> mkdir src\config, src\controllers, src\routes, src\middleware, public

Directorio: C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api\src

Mode                LastWriteTime         Length Name
----                -
d-----         10/04/2026   7:40 p. m.         config
d-----         10/04/2026   7:40 p. m.         controllers
d-----         10/04/2026   7:40 p. m.         routes
d-----         10/04/2026   7:40 p. m.         middleware

Directorio: C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api

Mode                LastWriteTime         Length Name
----                -
d-----         10/04/2026   7:40 p. m.         public
```

Paso 3 — Inicialización e instalación de dependencias

Descripción Se inicializa el proyecto con npm init y se instalan las dependencias: express, mysql2, bcryptjs, jsonwebtoken, cors, dotenv y nodemon. Se obtienen 121 módulos instalados sin vulnerabilidades.

Acción `npm init -y && npm install express mysql2 bcryptjs jsonwebtoken cors dotenv && npm install --save-dev nodemon`

Captura 3

CAPTURA 3 — INICIALIZACIÓN E INSTALACIÓN DE DEPENDENCIAS

```
PS C:\Users\JULIAN\PROYECTOS\ADS\leads-auth-api> npm init -y
>> npm install express mysql2 bcryptjs jsonwebtoken cors dotenv
● >> npm install --save-dev nodemon
Wrote to C:\Users\JULIAN\PROYECTOS\ADS\leads-auth-api\package.json:
```

```
{
  "name": "leads-auth-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

```
added 94 packages, and audited 95 packages in 4s
```

```
27 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities
```

```
added 25 packages, and audited 120 packages in 2s
```

```
32 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities
```

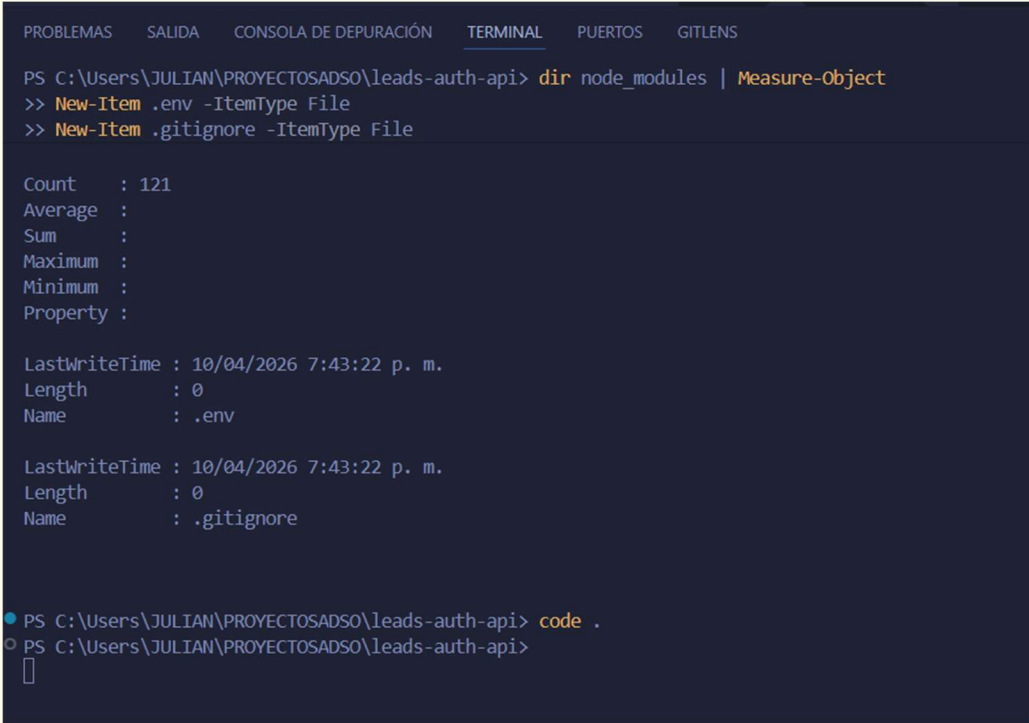

Paso 4 — Configuración de variables de entorno

Descripción Se crea el archivo .env con las credenciales de la base de datos, el puerto del servidor y la clave secreta JWT. El archivo .gitignore excluye .env y node_modules del repositorio.

Acción `New-Item .env -ItemType File` | Editar con: `PORT`, `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`

Captura 4

CAPTURA 4 — CONFIGURACIÓN DE VARIABLES DE ENTORNO



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  GITLENS

PS C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api> dir node_modules | Measure-Object
>> New-Item .env -ItemType File
>> New-Item .gitignore -ItemType File

Count      : 121
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :

LastWriteTime : 10/04/2026 7:43:22 p. m.
Length       : 0
Name         : .env

LastWriteTime : 10/04/2026 7:43:22 p. m.
Length       : 0
Name         : .gitignore

● PS C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api> code .
○ PS C:\Users\JULIAN\PROYECTOSADSO\leads-auth-api>
```

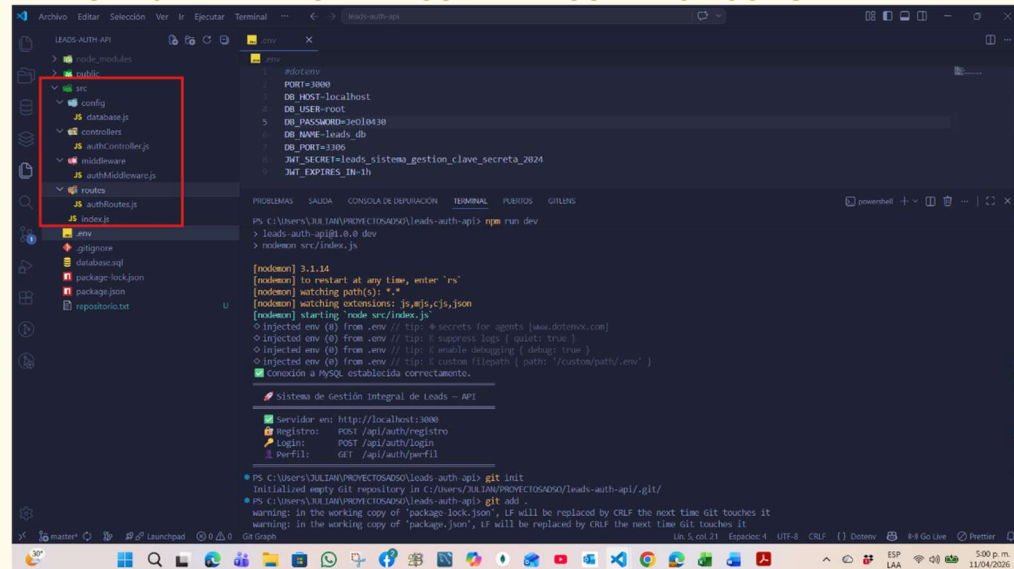
Paso 5 — Creación de los archivos de código fuente

Descripción Se implementan los 5 módulos del proyecto en VSC: database.js (conexión MySQL), authController.js (lógica de registro y login), authMiddleware.js (verificación JWT), authRoutes.js (endpoints) e index.js (servidor Express). **Cada archivo incluye comentarios técnicos de documentación.**

Acción VSC: crear src/config/database.js | src/controllers/authController.js | src/middleware/authMiddleware.js | src/routes/authRoutes.js | src/index.js

Captura 5

CAPTURA 5 — CREACIÓN DE LOS ARCHIVOS DE CÓDIGO FUENTE



The screenshot shows the Visual Studio Code interface with the 'LEADS-Auth API' project. The Explorer sidebar on the left shows the file structure: src > config > database.js, src > controllers > authController.js, src > middleware > authMiddleware.js, src > routes > authRoutes.js, and src > index.js. The main editor displays the 'index.js' file, which contains the following code:

```
#!/usr/bin/env node
import { createServer } from 'http';
import { createApp } from './src/index.js';

const app = createApp();
const server = createServer(app);

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

The terminal at the bottom shows the command 'npm run dev' being executed, which starts the development server. The output includes the following information:

- Node.js version: 16.14.0
- npm version: 8.3.1
- Project name: leads-auth-api
- Project path: C:\Users\JUAN\PROJECTOS\LEADS-Auth API\src
- Project type: javascript
- Project description: Sistema de Gestión Integral de Leads - API
- Project keywords: leads, auth, api
- Project license: MIT
- Project repository: https://github.com/JUAN/PROJECTOS/LEADS-Auth API
- Project scripts: start: node src/index.js, dev: npm run start -- --mode development
- Project dependencies: express: ^4.17.1, mysql: ^2.18.0, bcrypt: ^5.0.1, jwt: ^8.5.1, dotenv: ^16.0.0, cors: ^2.8.5, helmet: ^5.0.2, morgan: ^1.10.0, winston: ^3.3.3, axios: ^0.27.0, lodash: ^4.17.21, uuid: ^8.3.2, nodemon: ^2.0.15, dotenv-cli: ^4.0.0
- Project devDependencies: nodemon: ^2.0.15, dotenv-cli: ^4.0.0
- Project warnings: warning: in the working copy of 'package-lock.json', it will be replaced by CRLF the next time git touches it

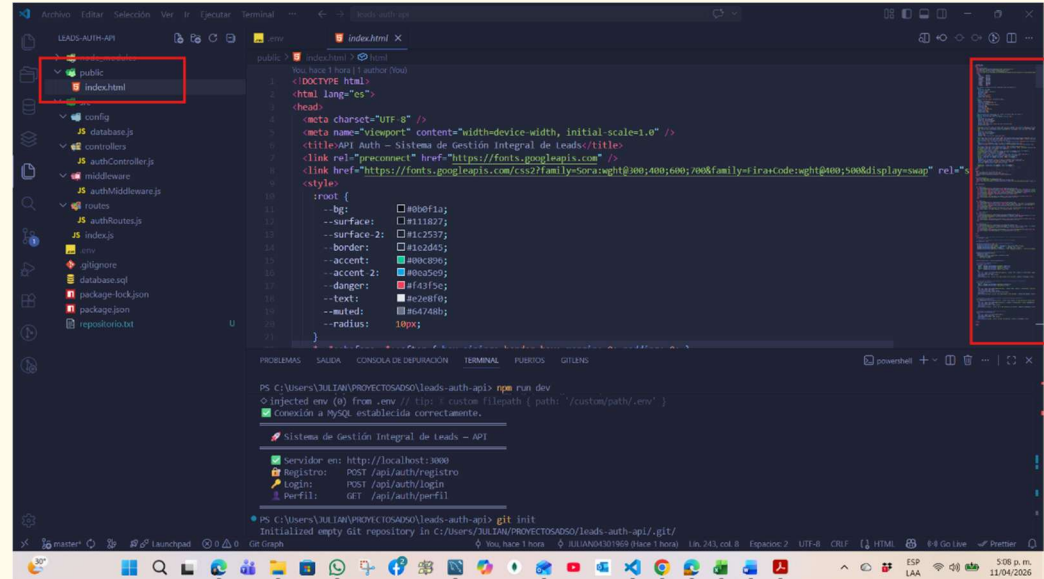
Paso 6 — Creación de la interfaz web de prueba

Descripción Se implementa public/index.html con una interfaz web que permite probar los cuatro endpoints del servicio directamente desde el navegador, sin necesidad de herramientas externas como Postman.

Acción VSC: crear public/index.html con formularios de registro, login, perfil y estado del servidor

Captura 6

CAPTURA 6 — CREACIÓN DE LA INTERFAZ WEB DE PRUEBA



Formularios de registro:

```
114 <!-- REGISTRO -->
115 <div class="card">
116   <div class="card-title"><div class="icon">👤</div>Registro de usuario</div>
117   <div class="field"><label>Nombre completo</label><input id="reg-nombre" type="text" placeholder="Ej. Juan Pérez" /></div>
118   <div class="field"><label>Correo electrónico</label><input id="reg-email" type="email" placeholder="correo@ejemplo.com" /></div>
119   <div class="field"><label>Contraseña</label><input id="reg-password" type="password" placeholder="Mínimo 6 caracteres" /></div>
120   <div class="field"><label>Rol</label><input id="reg-rol" type="text" value="asesor" /></div>
121   <button class="btn btn-primary" onclick="registrar()">Crear cuenta</button>
122   <div class="divider"></div>
123   <div class="status"><span class="dot" id="reg-dot"></span><span id="reg-status">En espera</span></div>
124   <div class="response" id="reg-response">// La respuesta aparecerá aquí</div>
125 </div>
126
```

Login:

```
127 <!-- LOGIN -->
128 <div class="card">
129   <div class="card-title"><div class="icon">🔑</div>Inicio de sesión</div>
130   <div class="field"><label>Correo electrónico</label><input id="login-email" type="email" value="admin@leads.com" /></div>
131   <div class="field"><label>Contraseña</label><input id="login-password" type="password" value="Admin2024*" /></div>
132   <button class="btn btn-primary" onclick="iniciarSesion()">Iniciar sesión</button>
133   <div class="divider"></div>
134   <div class="status"><span class="dot" id="login-dot"></span><span id="login-status">En espera</span></div>
135   <div class="response" id="login-response">// La respuesta aparecerá aquí</div>
136 </div>
137
```

Perfil Protegido:

```
138 <!-- PERFIL PROTEGIDO -->
139 <div class="card">
140   <div class="card-title"><div class="icon">👤</div>Ruta protegida – Perfil</div>
141   <p style="font-size:12px;color:var(--muted);line-height:1.6;">Requiere token JWT válido obtenido en el inicio de sesión. Demuestr
142   <button class="btn btn-secondary" onclick="obtenerPerfil()">Consultar perfil</button>
143   <div class="divider"></div>
144   <div class="status"><span class="dot" id="perfil-dot"></span><span id="perfil-status">En espera</span></div>
145   <div class="response" id="perfil-response">// La respuesta aparecerá aquí</div>
146 </div>
147
```

Estado del Servidor:

```
147
148 <!-- ESTADO DEL SERVIDOR -->
149 <div class="card">
150   <div class="card-title"><div class="icon">🖥️</div>Estado del servidor</div>
151   <p style="font-size:12px;color:var(--muted);line-height:1.6;">Verifica que el servidor Express y la conexión a MySQL están operati
152   <button class="btn btn-secondary" onclick="verificarEstado()">Verificar estado</button>
153   <div class="divider"></div>
154   <div class="status"><span class="dot" id="estado-dot"></span><span id="estado-status">En espera</span></div>
155   <div class="response" id="estado-response">// La respuesta aparecerá aquí</div>
156 </div>
157
158 </div>
```

Petición POST al endpoint de registro:

```
185 // Petición POST al endpoint de registro
186 async function registrar() {
187   const body = {
188     nombre: document.getElementById('reg-nombre').value.trim(),
189     email: document.getElementById('reg-email').value.trim(),
190     password: document.getElementById('reg-password').value,
191     rol: document.getElementById('reg-rol').value.trim()
192   };
193   try {
194     const res = await fetch(`${API_BASE}/auth/registro`, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body) });
195     const data = await res.json();
196     mostrarRespuesta('reg', data, res.ok);
197   } catch (err) {
198     mostrarRespuesta('reg', { error: 'No se pudo conectar con el servidor.', detalle: err.message }, false);
199   }
200 }
201
```

Petición POST al endpoint de login — guarda el token si la autenticación es satisfactoria:

```
202 // Petición POST al endpoint de login — guarda el token si la autenticación es satisfactoria
203 async function iniciarSesion() {
204   const body = {
205     email: document.getElementById('login-email').value.trim(),
206     password: document.getElementById('login-password').value
207   };
208   try {
209     const res = await fetch(`${API_BASE}/auth/login`, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body) });
210     const data = await res.json();
211     mostrarRespuesta('login', data, res.ok);
212     if (res.ok && data.token) actualizarTokenBar(data.token);
213   } catch (err) {
214     mostrarRespuesta('login', { error: 'No se pudo conectar con el servidor.', detalle: err.message }, false);
215   }
216 }
217
```

Petición GET a la ruta protegida incluyendo el token en el encabezado Authorization:

```
218 // Petición GET a la ruta protegida incluyendo el token en el encabezado Authorization You, hace 1 hora • feat: servicio web
219 async function obtenerPerfil() {
220   if (!tokenActivo) { mostrarRespuesta('perfil', { exito: false, mensaje: 'No hay token activo. Inicie sesión primero.' }, false);
221   try {
222     const res = await fetch(`${API_BASE}/auth/perfil`, { headers: { 'Authorization': `Bearer ${tokenActivo}` } });
223     const data = await res.json();
224     mostrarRespuesta('perfil', data, res.ok);
225   } catch (err) {
226     mostrarRespuesta('perfil', { error: 'No se pudo conectar con el servidor.', detalle: err.message }, false);
227   }
228 }
229 }
```

Petición GET al endpoint de health check del servidor:

```
230 // Petición GET al endpoint de health check del servidor You, hace 1 hora • feat: servicio web de autenticacion GA7-2205010
231 async function verificarEstado() {
232   try {
233     const res = await fetch(`${API_BASE}/estado`);
234     const data = await res.json();
235     mostrarRespuesta('estado', data, res.ok);
236   } catch (err) {
237     mostrarRespuesta('estado', { error: 'Servidor no disponible.', detalle: err.message }, false);
238   }
239 }
```

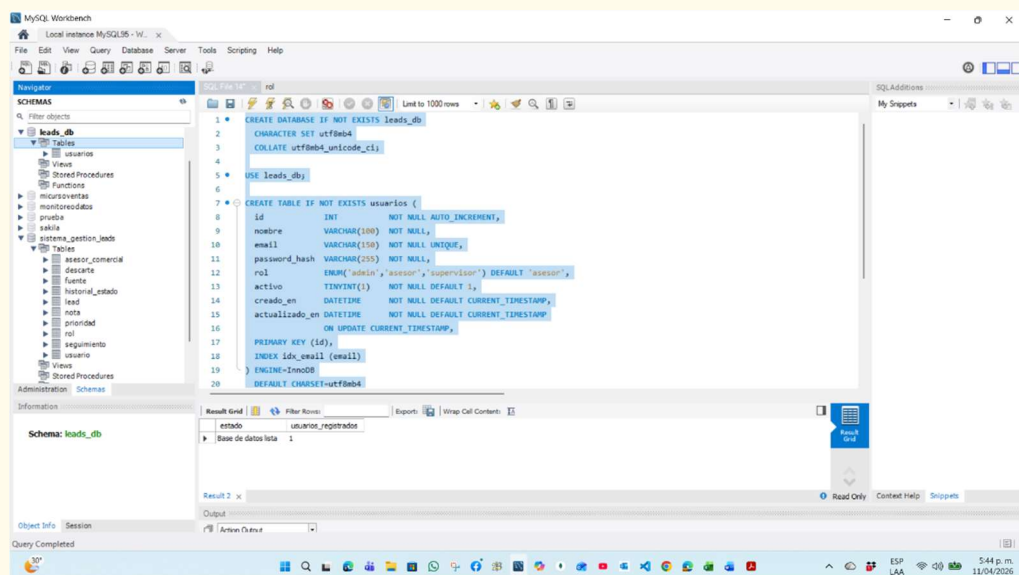
Paso 7 — Creación de la base de datos en MySQL Workbench

Descripción Se ejecuta el script database.sql en MySQL Workbench. El script crea la base de datos leads_db con codificación UTF-8, la tabla usuarios con todos sus campos y un usuario administrador de prueba con contraseña hasheada por bcrypt.

Acción MySQL Workbench → Ctrl+T → pegar database.sql → Ctrl+Shift+Enter → Query Completed

Captura 7

CAPTURA 7 — CREACIÓN DE LA BASE DE DATOS EN MYSQL WORKBENCH



Paso 8 — Arranque del servidor Node.js

Descripción Se ejecuta npm run dev. El servidor confirma la conexión exitosa a MySQL y queda escuchando en http://localhost:3000 con todos los endpoints disponibles.

Acción npm run dev → ☒ Conexión a MySQL establecida correctamente. → ☒
Servidor en: http://localhost:3000

Captura 8

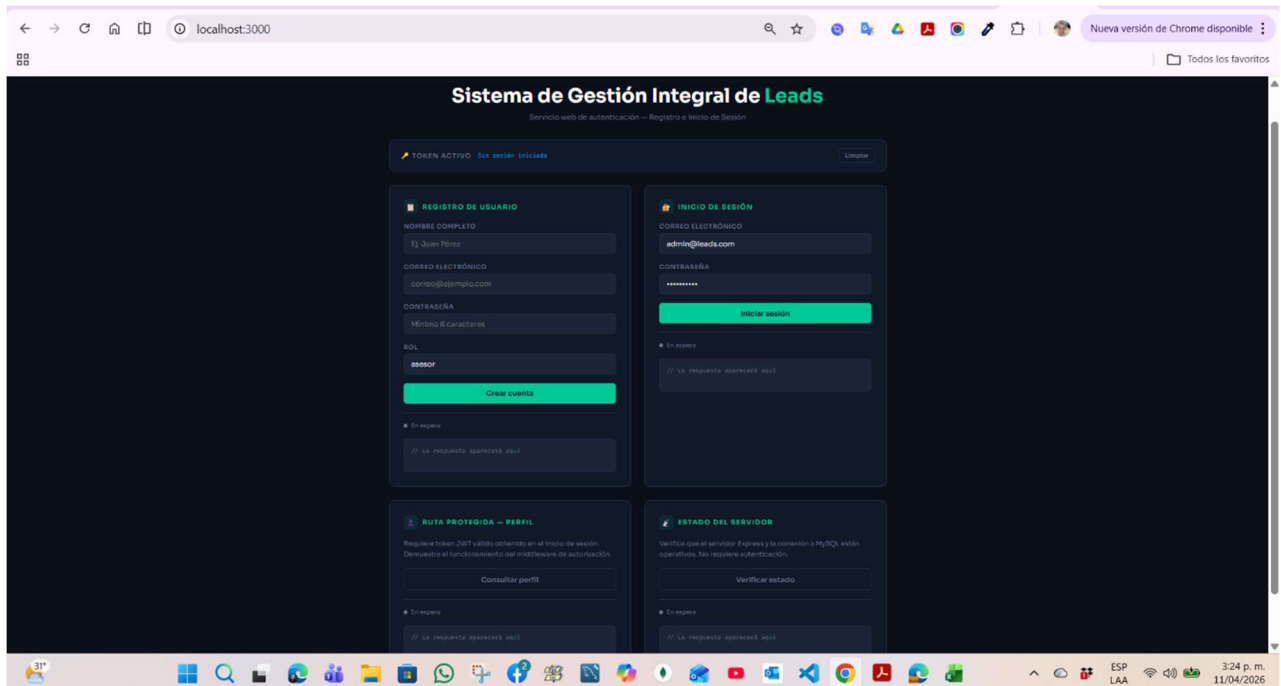
CAPTURA 8 — ARRANQUE DEL SERVIDOR NODE.JS

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  GITLENS

PS C:\Users\JULIAN\PROYECTOS\ADS\leads-auth-api> npm run dev
◇ injected env (0) from .env // tip: # custom filepath { path: '/custom/path/.env' }
✓ Conexión a MySQL establecida correctamente.

🚀 Sistema de Gestión Integral de Leads – API

✓ Servidor en: http://localhost:3000
🔑 Registro: POST /api/auth/registro
🔑 Login: POST /api/auth/login
👤 Perfil: GET /api/auth/perfil
```



Paso 9 — Prueba 1 — Estado del servidor

Descripción Se accede a `http://localhost:3000` y se pulsa "Verificar estado". El servidor responde con HTTP 200 confirmando que la API y la base de datos están operativas.

Acción `GET /api/estado → { "exito": true, "mensaje": "API del Sistema de Gestión Integral de Leads operativa." }`

Captura 9

CAPTURA 9 — PRUEBA 1 — ESTADO DEL SERVIDOR



Paso 10 — Prueba 2 — Registro de usuario nuevo

Descripción	Se completa el formulario de registro con nombre "JULIAN OCAMPO", correo julianenriqueocampolopez@gmail.com y rol asesor. El servidor responde HTTP 201 con los datos del usuario creado, sin exponer el hash de la contraseña.
Acción	POST /api/auth/registro → { "exito": true, "mensaje": "Usuario registrado exitosamente.", "datos": { "id": 3, ... } }

Captura 10

CAPTURA 10 — PRUEBA 2 — REGISTRO DE USUARIO NUEVO

REGISTRO DE USUARIO

NOMBRE COMPLETO

JULIAN OCAMPO

CORREO ELECTRÓNICO

julianenriqueocampolopez@gmail.com

CONTRASEÑA

.....

ROL

asesor

Crear cuenta

● Respuesta recibida ✓

```
{
  "exito": true,
  "mensaje": "Usuario registrado exitosamente.",
  "datos": {
    "id": 3,
    "nombre": "JULIAN OCAMPO",
    "email": "julianenriqueocampolopez@gmail.com",
    "rol": "asesor"
  }
}
```


Paso 11 — Prueba 3 — Inicio de sesión exitoso

Descripción Se inicia sesión con el usuario registrado. El servidor valida las credenciales, genera un token JWT firmado y responde con el mensaje de autenticación satisfactoria, el token y los datos básicos del usuario.

Acción POST /api/auth/login → { "exito": true, "mensaje": "✅ Autenticación satisfactoria.", "token": "eyJ..." }

Captura 11

CAPTURA 11 — PRUEBA 3 — INICIO DE SESIÓN EXITOSO

The screenshot shows a login form titled "INICIO DE SESIÓN" with a lock icon. It has two input fields: "CORREO ELECTRÓNICO" containing "julianenriqueocampolopez@gmail.com" and "CONTRASEÑA" with masked characters. A green "Iniciar sesión" button is below. A green status indicator shows "Respuesta recibida ✓". A code block displays the JSON response:

```
{
  "exito": true,
  "mensaje": "✅ Autenticación satisfactoria.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImYwIiwiaWF0IjpdWxpYW5lbnJpcXVlb2NhbXBvbG9wZCpAZ21haWwY29tIiwicm9sIjoieYXNlc29yIiwibm9tYnJlIjoieS1VMSUFOIE9DQU1QTyIsImhhdCI6MTc3NTkzOTc5MCwiZXhwIjo5Nzc1OTQzMzkwfQ.CGhN18VzQ7o-HYf0gKJiLLQGYIiTQEGFW8RVMqNkrtY",
  "usuario": {
    "id": 3,
    "nombre": "JULIAN OCAMPO",
  }
}
```

Paso 12 — Prueba 4 — Error en la autenticación

Descripción Se intenta iniciar sesión con una contraseña incorrecta. El servidor responde HTTP 401 con el mensaje de error en la autenticación, sin revelar si el email existe o no en el sistema.

Acción `POST /api/auth/login (contraseña incorrecta) → { "exito": false, "mensaje": "Error en la autenticación. Credenciales incorrectas." }`

Captura 12

CAPTURA 12 — PRUEBA 4 — ERROR EN LA AUTENTICACIÓN



Paso 13 — Prueba 5 — Ruta protegida con token JWT

Descripción Con el token obtenido en el login, se consulta la ruta protegida GET /api/auth/perfil. El middleware verifyToken valida el token y el servidor devuelve los datos completos del usuario autenticado.

Acción GET /api/auth/perfil → Authorization: Bearer <token> → { "exito": true, "datos": { "id": 3, "nombre": "JULIAN OCAMPO", ... } }

Captura 13

CAPTURA 13 — PRUEBA 5 — RUTA PROTEGIDA CON TOKEN JWT



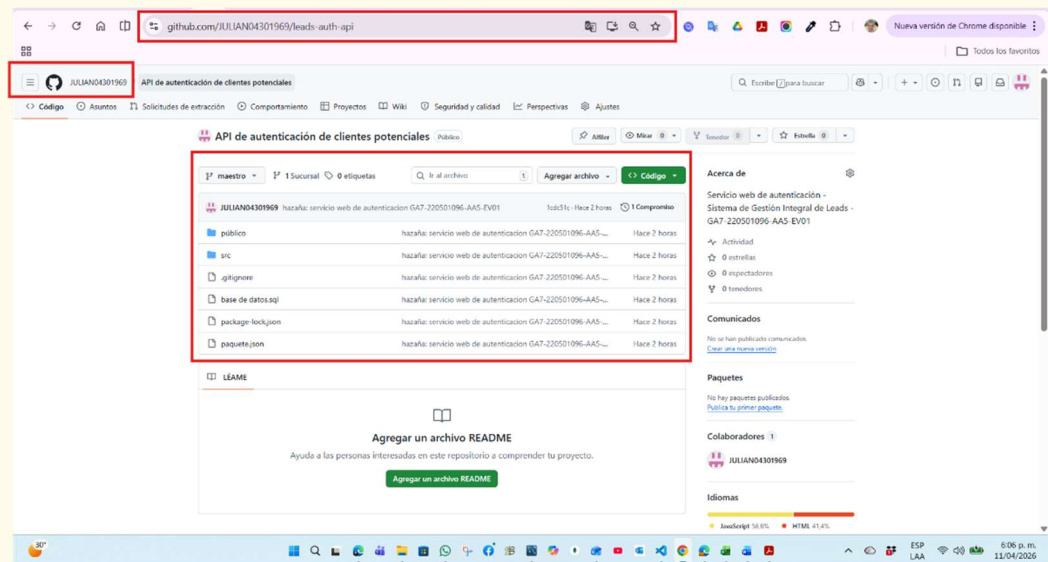
Paso 14 — Publicación en GitHub

Descripción Se inicializa el repositorio Git, se realiza el primer commit con los 10 archivos del proyecto y se publica en GitHub. El repositorio queda público y accesible con el historial de cambios documentado.

Acción `git init && git add . && git commit -m "feat: servicio web de autenticacion GA7-220501096-AA5-EV01" && git push -u origin master`

Captura 14

CAPTURA 14 — PUBLICACIÓN EN GITHUB



<https://github.com/JULIAN04301969/leads-auth-api.git>

6. Medidas de seguridad implementadas

Medida	Implementación
Hash bcrypt de contraseñas	Las contraseñas se procesan con bcrypt (costo 10) antes de persistirse. Nunca se almacena ni devuelve la contraseña en texto plano.
Tokens JWT firmados	Los tokens se firman con clave secreta almacenada en variables de entorno y expiran en 1 hora.
Mensajes de error genéricos	Ante credenciales inválidas siempre se responde con el mismo mensaje, sin indicar si el email existe o no.
Variables de entorno	Todas las credenciales se almacenan en .env, excluido del repositorio mediante .gitignore.
Validación de entrada	Se validan campos obligatorios, formato de email y longitud mínima de contraseña antes de procesar.
Pool de conexiones	Se usa pool MySQL con límite de 10 conexiones para evitar saturar el servidor de base de datos.

7. Conclusiones

El desarrollo de este servicio web de autenticación permitió aplicar de forma integrada los conceptos fundamentales de las API REST, la seguridad en el manejo de credenciales, la arquitectura en capas y el uso de herramientas de versionamiento de código, todos ellos alineados con los criterios de evaluación del instrumento GA7-220501096-AA5-EV01.

Se implementaron satisfactoriamente los dos servicios requeridos por la evidencia: el registro de usuarios, que persiste la información de forma segura mediante hashing bcrypt, y el inicio de sesión, que valida las credenciales y responde con un mensaje de autenticación satisfactoria o de error según corresponda, cubriendo así el 50% de los indicadores de logro con los endpoints principales.

La incorporación de JSON Web Tokens como mecanismo de autorización y la implementación de un middleware de verificación garantizan que las rutas privadas del sistema solo sean accesibles por usuarios autenticados, cumpliendo con el tercer indicador de validaciones de verificación correctamente.

La publicación del código fuente en GitHub con un commit documentado y un repositorio público evidencia el uso profesional de herramientas de versionamiento, satisfaciendo el cuarto y último indicador de la lista de chequeo.

La separación del código en capas independientes — rutas, controladores, middleware y configuración — junto con los comentarios técnicos incluidos en cada módulo, garantizan que el servicio sea mantenible, escalable y comprensible por otros desarrolladores, constituyendo una base sólida para las etapas siguientes del proyecto Sistema de Gestión Integral de Leads.

8. Bibliografía

MDN Web Docs. (2024). HTTP — HyperText Transfer Protocol. Mozilla Developer Network. <https://developer.mozilla.org/es/docs/Web/HTTP>

OpenJS Foundation. (2024). Node.js Documentation v22. <https://nodejs.org/docs/latest/api/>

Express.js. (2024). Express 4.x — API Reference. <https://expressjs.com/en/4x/api.html>

Auth0. (2024). Introduction to JSON Web Tokens. <https://jwt.io/introduction>

Provos, N. & Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference. <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>

MySQL. (2024). MySQL 9.5 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/9.5/en/>

GitHub. (2024). GitHub Docs — Getting started with Git. <https://docs.github.com/es/get-started>

SENA. (2024). Material de formación: Construcción de API. Actividad de Proyecto 7 — Fase 3 Ejecución. Sistema de Gestión de Aprendizaje SENA.

9. Listado de capturas de pantalla

Las siguientes capturas de pantalla documentan cada etapa del desarrollo e implementación del servicio web. Deben insertarse en los bloques correspondientes de la Sección 5.

No.	Nombre del archivo	Descripción del momento capturado
1	Captura 1 — Verificación del entorno	Salida de terminal con versiones de Node.js, npm, Git y MySQL confirmadas.
2	Captura 2 — Estructura del proyecto	Explorador de VSC mostrando la carpeta leads-auth-api con todas las subcarpetas creadas.
3	Captura 3 — Instalación de dependencias	Terminal con salida de npm install mostrando 121 módulos instalados sin vulnerabilidades.
4	Captura 4 — Archivo .env y package.json	VSC mostrando el archivo .env con las variables configuradas y package.json con los scripts.
5	Captura 5 — Archivos de código fuente	VSC mostrando los 5 archivos JS en sus carpetas respectivas con el código implementado.
6	Captura 6 — Interfaz web index.html	VSC mostrando el archivo public/index.html con la interfaz de prueba.
7	Captura 7 — Ejecución SQL en Workbench	MySQL Workbench con resultado: Base de datos lista 1 usuario registrado.
8	Captura 8 — Servidor en ejecución	Terminal VSC con npm run dev mostrando conexión MySQL exitosa y endpoints activos.
9	Captura 9 — Prueba 1: Estado del servidor	Interfaz web con respuesta exitosa del endpoint GET /api/estado.
10	Captura 10 — Prueba 2: Registro de usuario	Interfaz web con respuesta HTTP 201 del registro exitoso de JULIAN OCAMPO.
11	Captura 11 — Prueba 3: Login exitoso	Interfaz web con respuesta de autenticación satisfactoria y token JWT visible.
12	Captura 12 — Prueba 4: Error de autenticación	Interfaz web con respuesta HTTP 401 ante credenciales incorrectas.
13	Captura 13 — Prueba 5: Ruta protegida	Interfaz web con datos del perfil devueltos por la ruta GET /api/auth/perfil.
14	Captura 14 — Repositorio GitHub	Pantalla del repositorio https://github.com/JULIAN04301969/leads-auth-api con los archivos publicados.

Anexo — Declaración de uso de inteligencia artificial como herramienta de apoyo

Herramienta utilizada: Claude (Anthropic) — Modelo Claude Sonnet 4.6, acceso web claude.ai

Rol de la herramienta: Apoyo técnico y colaboración asistida bajo supervisión del aprendiz

Durante el desarrollo de la presente evidencia de desempeño, el aprendiz hizo uso del asistente de inteligencia artificial Claude, desarrollado por Anthropic, como herramienta de apoyo y colaboración en la construcción del servicio web de autenticación.

El asistente fue empleado como un colaborador técnico de consulta, cumpliendo funciones equivalentes a las de un tutor de programación o un entorno de documentación interactiva. Su participación se circunscribió a las siguientes tareas:

- Generación de la estructura base del código fuente de la API REST en Node.js con Express, tomando como referencia los conceptos trabajados en el material de formación "Construcción de API" de la Actividad de Proyecto 7, Fase 3 de Ejecución.
- Orientación en la configuración del entorno de desarrollo, la instalación de dependencias y la resolución de errores de conexión a la base de datos durante la etapa de implementación.
- Redacción de comentarios técnicos de documentación dentro del código fuente, coherentes con las buenas prácticas de desarrollo de software abordadas en las sesiones de formación en línea.
- Apoyo en la estructuración del presente documento, cuyo contenido fue revisado, ajustado y validado por el aprendiz a partir de los materiales de formación y las indicaciones del instructor.

En ningún caso la herramienta reemplazó el criterio, la supervisión ni la toma de decisiones del aprendiz. Cada fragmento de código generado fue comprendido, revisado y ejecutado por el aprendiz en su propio entorno de desarrollo. Los conceptos aplicados — API REST, autenticación JWT, hashing de contraseñas, arquitectura en capas y versionamiento con Git — fueron trabajados previamente en el programa de formación y constituyen la base conceptual sobre la cual se orientó el uso del asistente.

El uso de esta herramienta se declara en cumplimiento del principio de transparencia académica y con el propósito de documentar de forma honesta el proceso de aprendizaje y construcción de la evidencia.

Referencia: Anthropic. (2026). Claude — AI Assistant. <https://claude.ai>